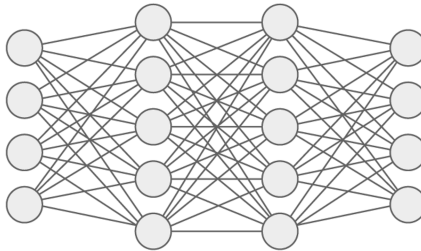


Deep Learning and Artificial Neural Networks

Hyperparameter Tuning, Batch Normalization



Contents

- 1 Hyperparameter Tuning
- 2 Practical Considerations
- 3 Batch Normalization
- 4 How Batch Normalization Works
- 5 Benefits of Batch Normalization

Why Hyperparameter Tuning Matters

- Deep learning models involve setting many hyperparameters.
- Some hyperparameters are more important than others.
- Hyperparameters:
 - **Most important:** Learning rate (α)
 - **Second importance:** Momentum term (β), mini-batch size, number of hidden units
 - **Third importance:** Number of layers, learning rate decay
 - **Least important:** Adam parameters ($\beta_1, \beta_2, \epsilon$) – rarely tuned
- Efficient hyperparameter tuning can significantly improve model performance.

Grid Search vs. Random Search

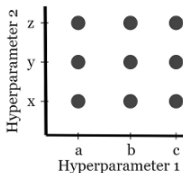
Grid Search:

- Traditional grid search explores hyperparameters systematically but inefficiently. For example: If one hyperparameter (e.g., α) matters much more than another (e.g., ϵ), grid search wastes resources.
- May waste evaluations on less important parameters.

Grid Search

Pseudocode

```
Hyperparameter_One = [a, b, c]  
Hyperparameter_Two = [x, y, z]
```



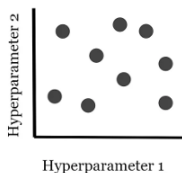
Random Search:

- Samples values randomly.
- Random sampling explores a wider range of values for the most important hyperparameters.
- **More effective for deep learning.**

Random Search

Pseudocode

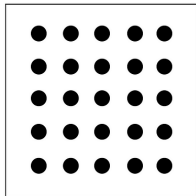
```
Hyperparameter_One = random.num(range)  
Hyperparameter_Two = random.num(range)
```



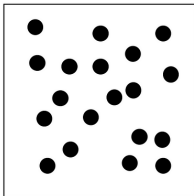
Coarse-to-Fine Approach

- Start with a coarse random sample across the entire hyperparameter space.
- Identify regions where the best-performing hyperparameters lie.
- Zoom into these regions and perform a finer-grained search.
- Example:
 - Initial random sample finds a promising region.
 - Focus subsequent samples within this region for better results.
- This approach balances exploration and exploitation effectively.

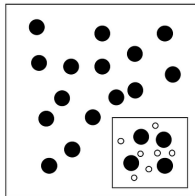
Grid Search



Grid Search at Random



Coarse to Fine



Practical Tips

- **Use random search** rather than grid search.
- Focus on tuning **learning rate (α)** first.
- Consider **coarse-to-fine search** for better efficiency.
- Optimize for performance on the validation set.

Efficient hyperparameter tuning can save significant time and computational resources.

Why Sampling Scale Matters

- Random sampling is more efficient than grid search.
- However, "random" doesn't always mean "uniform."
- Choosing the right scale ensures better exploration of hyperparameters.

Why Sampling Scale Matters

- Random sampling is more efficient than grid search.
- However, "random" doesn't always mean "uniform."
- Choosing the right scale ensures better exploration of hyperparameters.

When Uniform Sampling Works

- For hyperparameters like:
 - Number of hidden units ($n[l]$): e.g., range [50, 100].
 - Number of layers (L): e.g., range [2, 4].
- Uniform sampling is reasonable in these cases.
- Example: Sampling uniformly between 2 and 4 for L works well.

Why Sampling Scale Matters

- Random sampling is more efficient than grid search.
- However, "random" doesn't always mean "uniform."
- Choosing the right scale ensures better exploration of hyperparameters.

When Uniform Sampling Works

- For hyperparameters like:
 - Number of hidden units ($n[l]$): e.g., range [50, 100].
 - Number of layers (L): e.g., range [2, 4].
- Uniform sampling is reasonable in these cases.
- Example: Sampling uniformly between 2 and 4 for L works well.

When Logarithmic Sampling is Better

- For hyperparameters like learning rate (α):
 - Range: [0.0001, 1].
 - Uniform sampling would focus too much on higher values (e.g., 0.1 to 1).
- Solution: Sample on a **logarithmic scale**.
- Example: Define $r = -4 \cdot \text{np.random.rand}()$. Set $\alpha = 10^r$, where $r \in [-4, 0]$.

Sampling Beta for Exponentially Weighted Averages

- Suppose $\beta \in [0.9, 0.999]$:
 - Linear sampling is inefficient because small changes in β near 1 have large effects.
 - Instead, sample $1 - \beta$ on a logarithmic scale.
- Steps:
 - 1 Define range for $1 - \beta$: $[0.1, 0.001]$.
 - 2 Convert to log scale: $[-1, -3]$.
 - 3 Sample r uniformly from $[-3, -1]$.
 - 4 Set $1 - \beta = 10^r$ and $\beta = 1 - 10^r$.

Why Logarithmic Sampling is Effective

Sensitivity Near 1

- When β is close to 1:
 - Small changes in β lead to large changes in behavior.
 - Formula: $\frac{1}{1-\beta}$ is highly sensitive.
- Logarithmic sampling ensures dense exploration near-critical regions.
- Example:
- $\beta = 0.999$ vs. $\beta = 0.9995$: significant impact.

Final Tips for Hyperparameter Search

- Deep learning applications vary widely.
- Intuitions about hyperparameters may not transfer across domains.
- Regularly reevaluate hyperparameters to adapt to changes in data or infrastructure.

Reevaluating Hyperparameters

- Over time, datasets evolve, algorithms improve, and computational resources change.
- Best hyperparameters can become stale due to:
 - Changes in data distribution.
 - Upgrades in hardware (e.g., faster GPUs).
 - Advances in model architectures.
- Recommendation: Retest hyperparameters every few months.

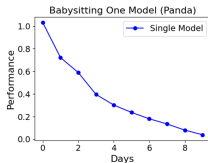
Two Major Strategies for Hyperparameter Search

- **Panda Approach (Babysitting One Model):**

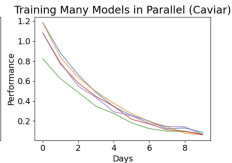
- Limited computational resources.
- Train one model at a time and adjust hyperparameters incrementally.
- Example: Nudge learning rate up or down daily based on performance.

- **Caviar Approach (Training Many Models in Parallel):**

- Abundant computational resources.
- Train multiple models with different hyperparameter settings simultaneously.
- Select the best-performing model after evaluation.



Panda



Caviar

Choosing the Right Approach

- Use the caviar approach if:
 - You have access to many CPUs/GPUs.
 - You can train multiple models in parallel.
- Use the panda approach if:
 - Computational resources are limited.
 - Training a single model takes a long time (e.g., large datasets).
- Hybrid Approach:
 - Start with the panda approach for initial tuning.
 - Transition to the caviar approach once resources allow.

Batch Normalization?

- Batch Normalization (BatchNorm) is a technique to improve the training of deep neural networks.
- It normalizes the pre-activation values within each layer, reducing internal covariate shift.
- Helps stabilize and accelerate training, making models more robust to hyperparameter choices.
- While originally motivated by reducing internal covariate shift, BatchNorm is now understood to improve optimization more generally.

Why Do We Need Batch Normalization?

- Problem: Internal covariate shift
 - Internal covariate shift refers to the phenomenon where the distribution of activations in a neural network changes during training as the model parameters are updated. This shift can make training slower and less stable because each layer must constantly adapt to the changing input distributions coming from the previous layer.
- Solution: Normalize the inputs to each layer during training.
 - Reduces the effect of changes in earlier layers on later layers.
 - Allows higher learning rates and reduces the sensitivity to initialization.

Implementation of Batch Normalization

- For a mini-batch of size m , compute the mean and variance of the **pre-activation** values $z^{(i)}$ for each layer:

$$\mu = \frac{1}{m} \sum_{i=1}^m z^{(i)}, \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^m (z^{(i)} - \mu)^2$$

- Normalize each input (pre-activation value) using the computed statistics:

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where ϵ is a small constant to avoid division by zero.

- Scale and shift the normalized input to restore representational power:

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

where γ and β are learnable parameters.

Batch Normalization: Training vs Inference

Training Phase

- After each mini-batch, update running (moving average) estimates:

$$\text{moving_mean} = \text{momentum} \cdot \text{moving_mean} + (1 - \text{momentum}) \cdot \mu_{\text{batch}}$$

$$\text{moving_var} = \text{momentum} \cdot \text{moving_var} + (1 - \text{momentum}) \cdot \sigma_{\text{batch}}^2$$

Inference Phase

- Do NOT use current batch statistics.
- Use stored running estimates:

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \text{moving_mean}}{\sqrt{\text{moving_var} + \epsilon}}$$

Why is this necessary?

- During inference, we may have a single sample or very small batch.
- Batch statistics would be unstable or meaningless.
- Running estimates provide consistent and reliable normalization.

Setting γ and β

- **Purpose:** Restore representational power after normalization.

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

- **Initialization:**

- γ is initialized to 1 (no scaling).
- β is initialized to 0 (no shifting).

- **Learning:**

- Updated via backpropagation alongside other model parameters.
- Allow the model to learn the optimal scale and shift for each feature.

- **Properties:**

- Per-feature parameters (one γ and β per channel).
- Automatically managed by deep learning frameworks.

Advantages of Batch Normalization

- Accelerates training:
 - Allows the use of higher learning rates.
 - Reduces the number of iterations required for convergence.
- Makes networks more robust:
 - Reduces the sensitivity to weight initialization.
 - Acts as a form of regularization, reducing overfitting.
- Simplifies hyperparameter tuning:
 - Reduces the need for careful tuning of learning rates and other hyperparameters.
- However, BatchNorm works poorly when the batch size is very small (noisy statistics).

Tensorflow Batch Normalization